GenAI SQL Refactoring Assistant – Milestone 5

Evaluation, Analysis & System Insights

Focus

Deep evaluation of optimized system

Techniques

RAG, FSP, CoT

Goal of Milestone 5

Evaluate System Performance

Evaluate system performance comprehensively

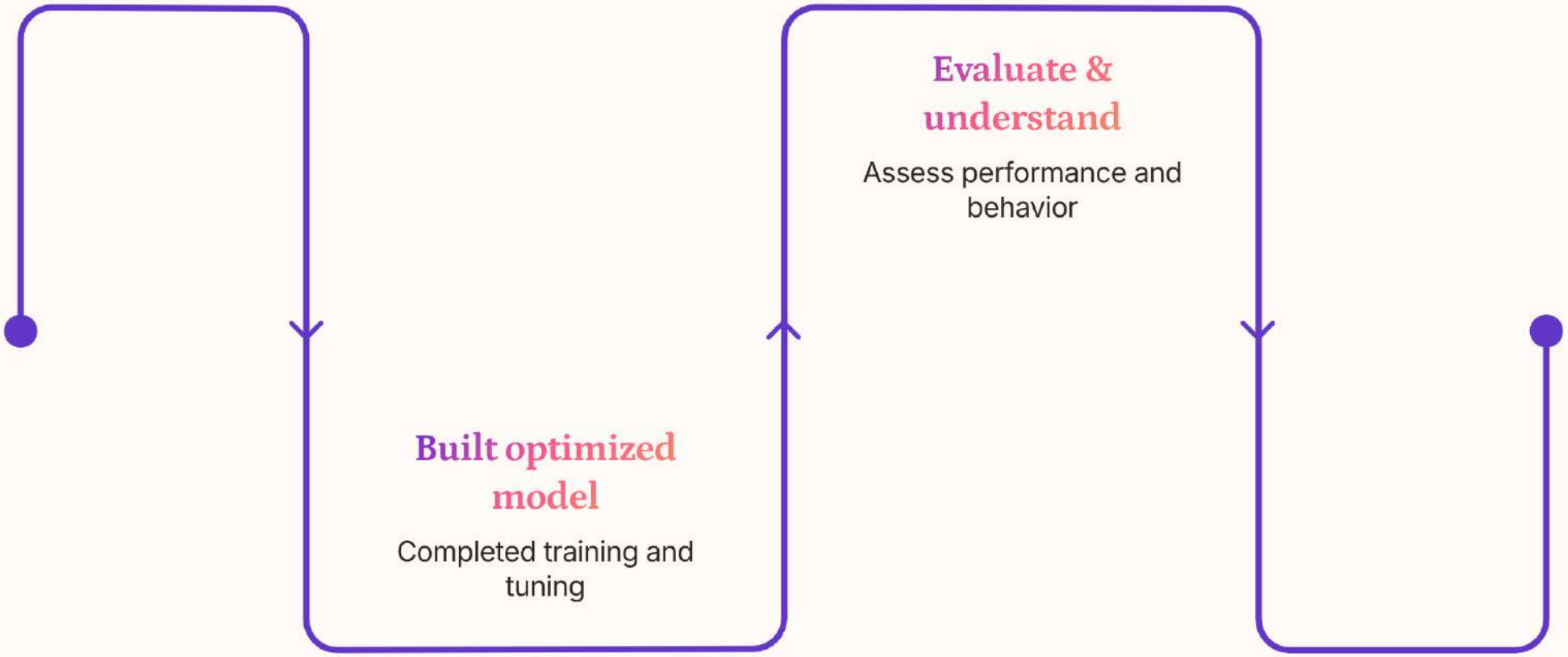
Compare Pipeline Variants

Compare multiple pipeline variants

Analyse

- Retrieval effectiveness
- Prompting strategies
- Reasoning impact

Transition from Milestone 4



👉 Shift from **model building → model analysis**

Evaluation Setup

Model

Best config (T5 + LoRA)

Pipeline

Full (RAG + FSP + CoT)

Inference

- Batched prediction
- Beam search (num_beams = 4)

Pipelines Compared

Pipeline	Description
Baseline	No context
RAG	Retrieval-based context
FSP	Few-shot prompting
CoT	Reasoning steps
Full	Combined system

Evaluation Metrics

Exact Match (EM%)

Strict correctness

BLEU

Similarity

Parse Validity

Syntax correctness

Token F1

Overlap

ROUGE-L

Sequence similarity

👉 Multi-metric evaluation ensures reliability

Training Convergence

Observed

- Loss: 6.31 → 0.12 (rapid drop)
- Validation loss stabilises early

Insight

👉 Model learns SQL transformations quickly

👉 Dataset is well-structured

Exact Match vs BLEU

Observed

- EM: fluctuates (81%–87%)
- BLEU: consistently high (~98–99%)

Insight

👉 **High BLEU ≠ correct SQL**

👉 Model outputs are similar but not exact

Parse Validity Issue

Observed

Parse Validity = 0

Insight

👉 Outputs look correct but fail execution checks

Possible Reasons

- Minor syntax differences
- Strict parsing rules

Impact of Regularisation (Dropout)

Observed

- EM improves (~92%)
- Smoother training

Insight

👉 Dropout prevents overfitting

👉 Improves generalisation

Overfitting Behaviour

Observed

- Training loss ↓ continuously
- Validation stabilises early

Insight

👉 Minimal overfitting

👉 Training beyond few epochs gives limited gains

Key Training Insights

- Fast convergence
- BLEU alone is misleading
- Exact Match is stricter
- Parse validity reveals hidden issues
- Regularisation improves robustness

Pipeline Performance and Comparison Insights

Baseline

Weakest (no context)

RAG

Major improvement

FSP

Improves pattern learning

CoT

Inconsistent gains

Full

Best overall

Core System Insight

👉 Performance improves with:

Context

Providing relevant background information to guide the model

Examples

Demonstrating patterns through few-shot prompting

Reasoning

Applying chain-of-thought to structure complex outputs

Context + Examples + Reasoning — the three pillars of improved performance.

Few-Shot Prompting (FSP)

Observed

- Examples consume large token space
- High overlap → better results

Insight

👉 **Example relevance > number of examples**

Choosing the right examples matters more than simply providing more of them.

FSP vs Query Complexity

Complex Queries	Simple Queries
Strong improvement observed with Few-Shot Prompting	Minimal effect from Few-Shot Prompting

👉 **FSP is useful for structural complexity**

Chain-of-Thought (CoT)

Observed

- Improves reasoning clarity
- Not always improving accuracy

Insight

👉 CoT helps interpretability

👉 Not a guaranteed performance boost

Retrieval Quality

Relevant Chunks	Irrelevant Chunks
→ Better output	→ Degraded output

👉 **Retrieval is a critical bottleneck**

Complexity-Based Performance

Baseline

Struggles with complex queries

RAG

Improves all complexity levels

Full Pipeline

Best for complex cases

👉 **Advanced techniques matter most for complex SQL**

Final Learnings and Insights

Final Learnings

- Context is the biggest improvement factor
- Retrieval quality directly impacts output
- Few-shot improves pattern learning
- CoT adds reasoning but inconsistently
- Multi-metric evaluation is essential

Final System

Best system includes:

- T5 + LoRA
- RAG
- Few-shot prompting
- Optional CoT

Final Insight

👉 Performance depends on:

Context Quality + Example Relevance + Reasoning Alignment